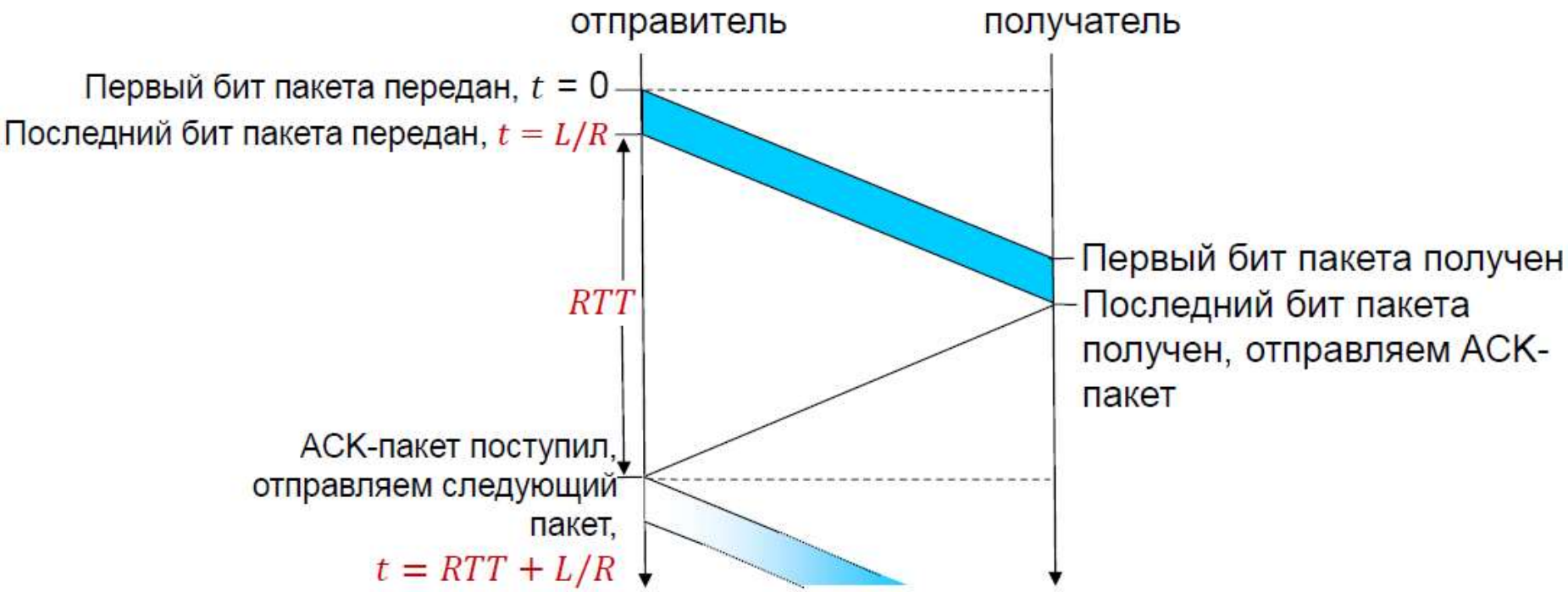
ТРАНСПОРТНЫЙ УРОВЕНЬ
Часть II

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Протокол rdt3.0: алгоритм с ожиданием подтверждений

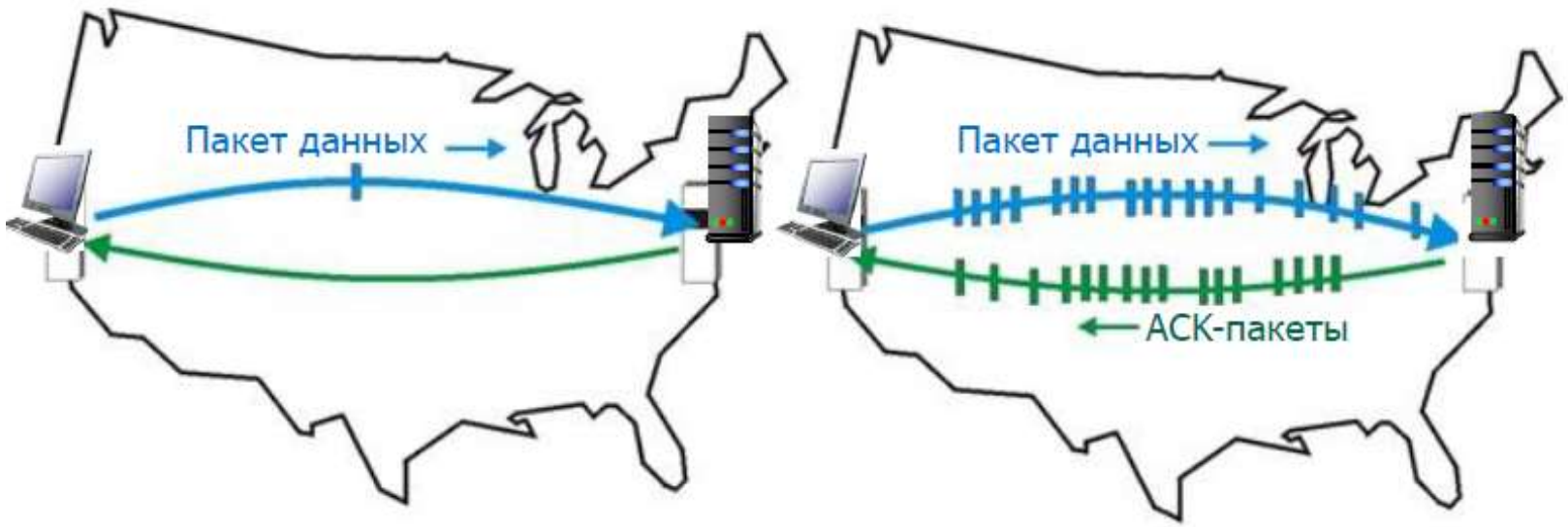


$$U_{\text{отправитель}} = \frac{L/R}{RTT + L/R} = \frac{0{,}008}{30{,}008} = 0{,}00027$$

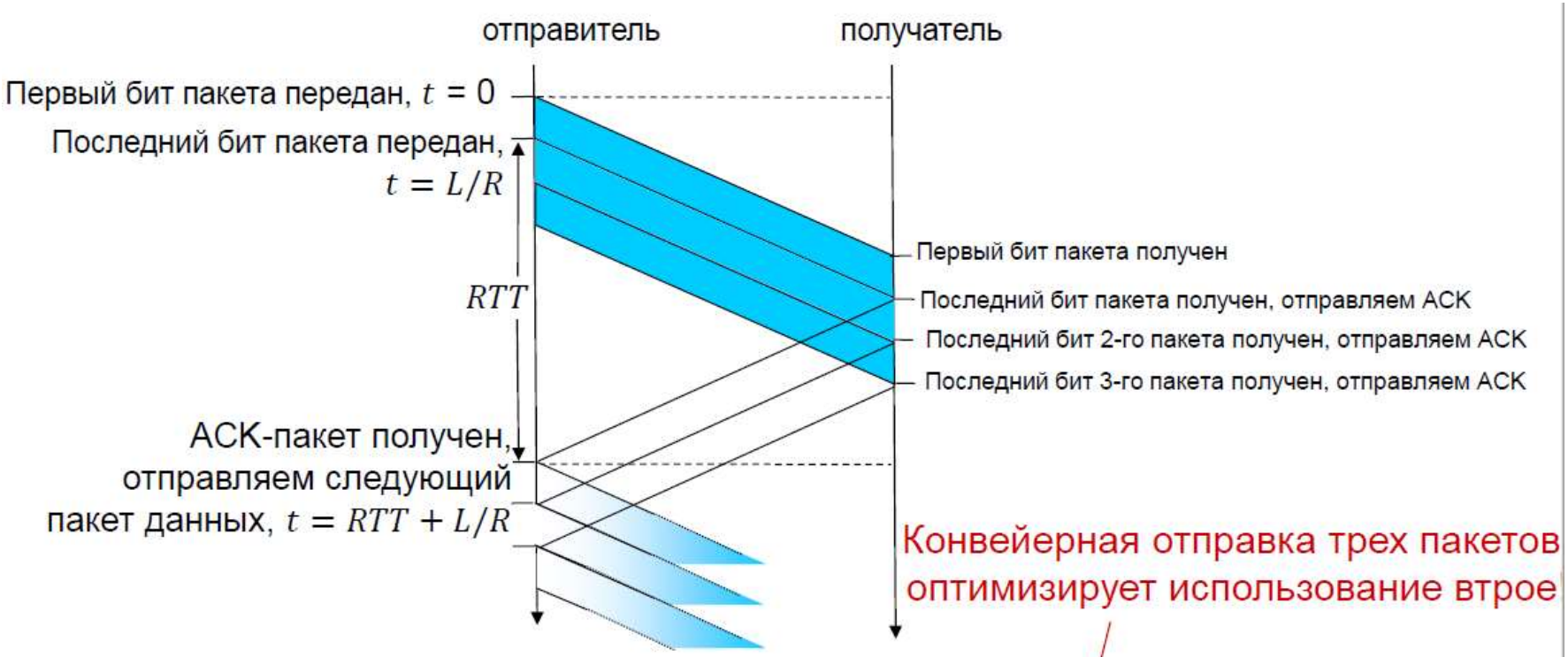
Протоколы с конвейеризацией

Конвейеризация: отправитель может одновременно отправлять несколько пакетов, не дожидаясь подтверждений

- Диапазон порядковых номеров должен быть увеличен
- Буферизация у отправителя и/или получателя
- Два базовых вида протокола с конвейеризацией: на N шагов назад и выборочное повторение



Конвейеризация: повышение использования канала



Конвейерная отправка трех пакетов оптимизирует использование втрое

$$U_{\text{отправитель}} = \frac{3L/R}{RTT + L/R} = \frac{0{,}0024}{30{,}008} = 0{,}00081$$

Протоколы с конвейеризацией: обзор

Возврат N шагов назад:

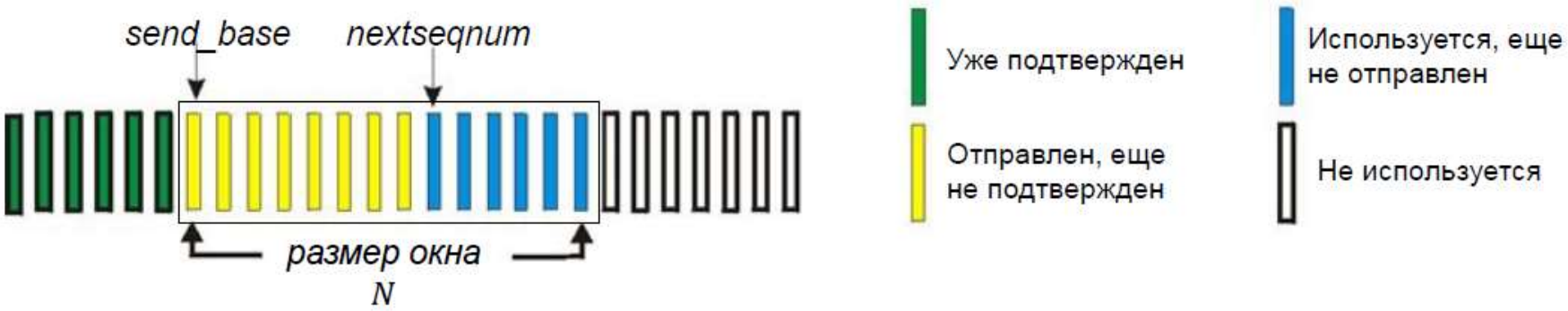
- Отправитель может одновременно предавать до N неподтвержденных пакетов
- Получатель отправляет только общий ACK-пакет
  - Не подтверждает пакеты в случае разрыва соединения
- Отправитель использует таймер наиболее раннего неподтвержденного пакета
  - Когда таймер истекает, **повторно передаются все пакеты**

Выборочное повторение:

- Отправитель может одновременно передавать до N неподтвержденных пакетов
- Получатель отправляет отдельные ACK-пакеты для каждого пакета
- Отправитель использует таймер для каждого неподтвержденного пакета
  - Когда таймер истекает, **повторно передаются только неподтвержденные пакеты**

Протокол Go-Back-N. Отправитель

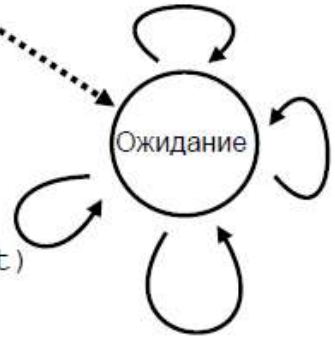
- k-разрядный порядковый номер в заголовке пакета
- «окно», вмещающее до N последовательных неподтвержденных пакетов
- ACK(n): получены ACK-пакеты на все пакеты данных до пакета с порядковым номером n – «общий ACK-пакет»
  - Может получать дублирующие ACK-пакеты
- Таймер для самого раннего передаваемого пакета
- timeout(n): повторная передача пакета **n** и всех пакетов с бОльшими порядковыми номерами в окне



GBN: расширенная FSM-схема отправителя

```
rdt_send(data)
_____________________
if (nextseqnum < base+N) {
    sndpkt[nextseqnum] = make_pkt(nextseqnum,data,chksum)
    udt_send(sndpkt[nextseqnum])
    if (base == nextseqnum)
       start_timer
    nextseqnum++
    }
else
  refuse_data(data)
```

```
Λ
_____
base=1
nextseqnum=1
```




```
timeout
_____________
start_timer
udt_send(sndpkt[base])
udt_send(sndpkt[base+1])
…
udt_send(sndpkt[nextseqnum-1])
```

```
rdt_rcv(rcvpkt)
   && corrupt(rcvpkt)
_____________________
```

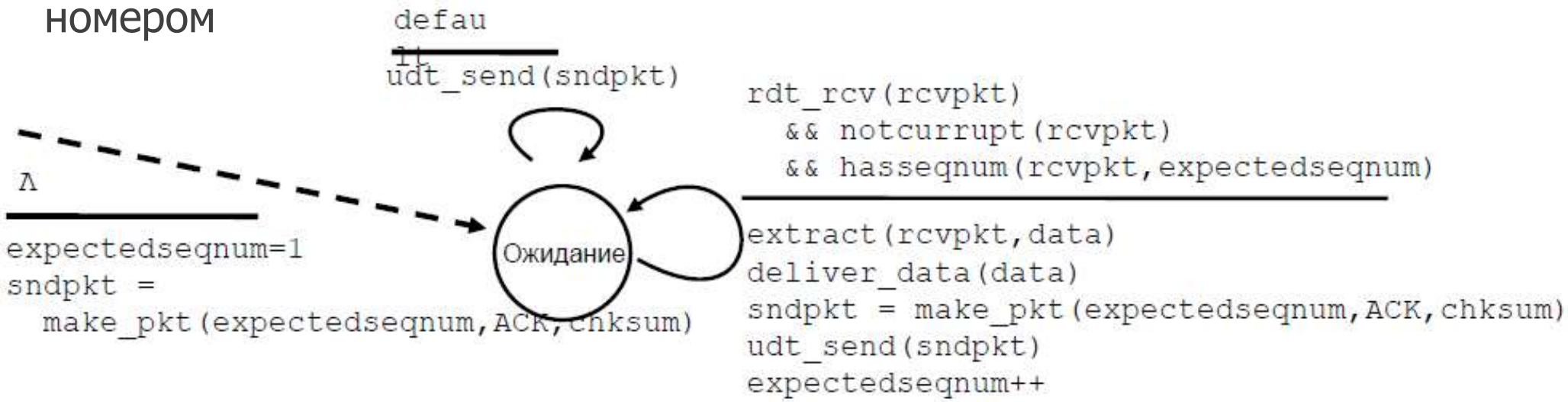
```
rdt_rcv(rcvpkt) &&
   notcorrupt(rcvpkt)
_____________________
base = getacknum(rcvpkt)+1
If (base == nextseqnum)
    stop_timer
  else
    start_timer
```

GBN: расширенная FSM-схема получателя

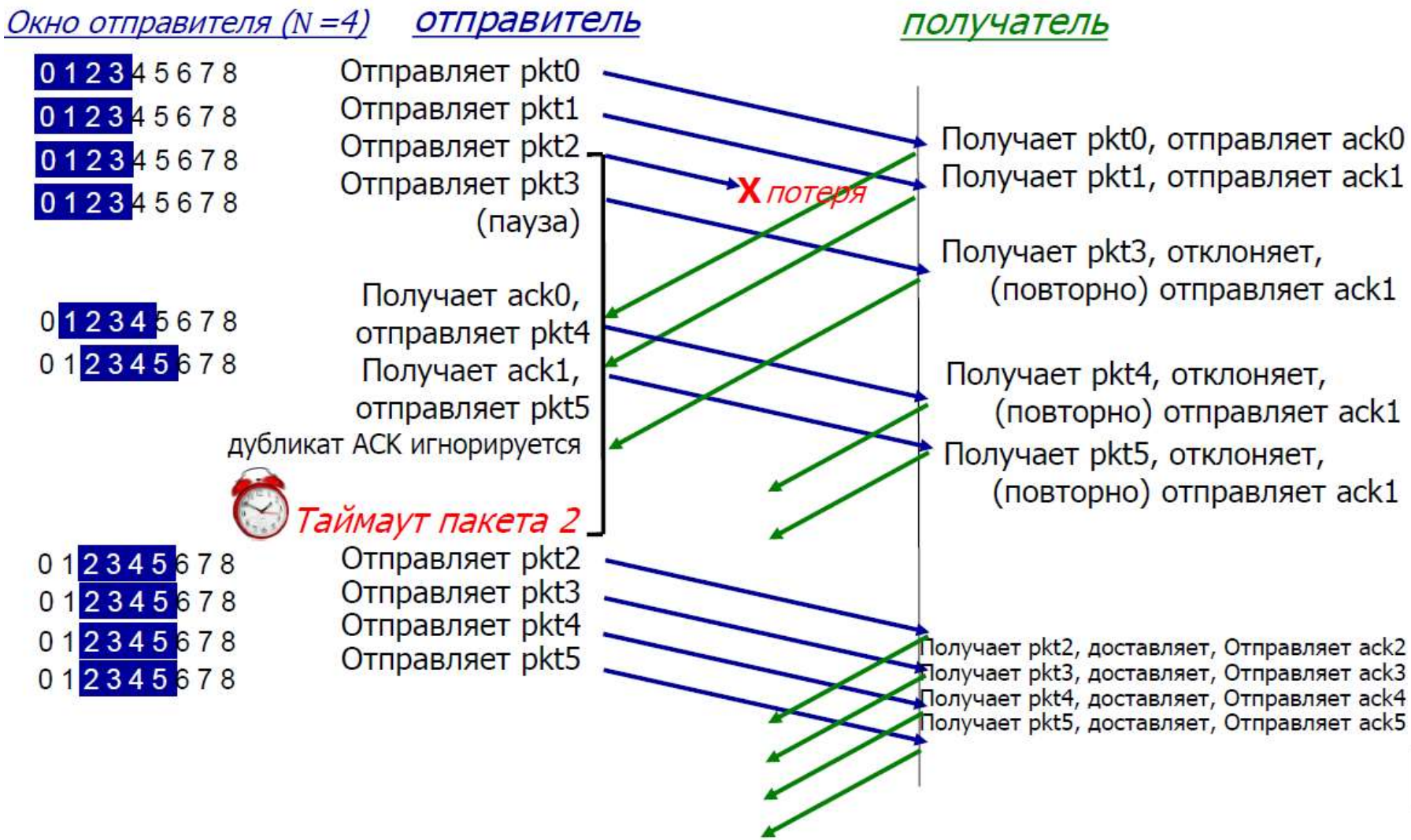
Только ACK: всегда отправляет ACK-пакет в ответ на корректно полученный пакет с наибольшим идущим по порядку порядковым номером

- Может создавать дублирующие ACK-пакеты
- Должен хранить только expectedseqnum (номер следующего по порядку пакета)

- Идущие не по порядку пакеты:
  - Отклоняются (нет буфера): нет буферизации на стороне получателя!
  - Повторное подтверждение пакета данных с наибольшим порядковым номером



GBN в действии



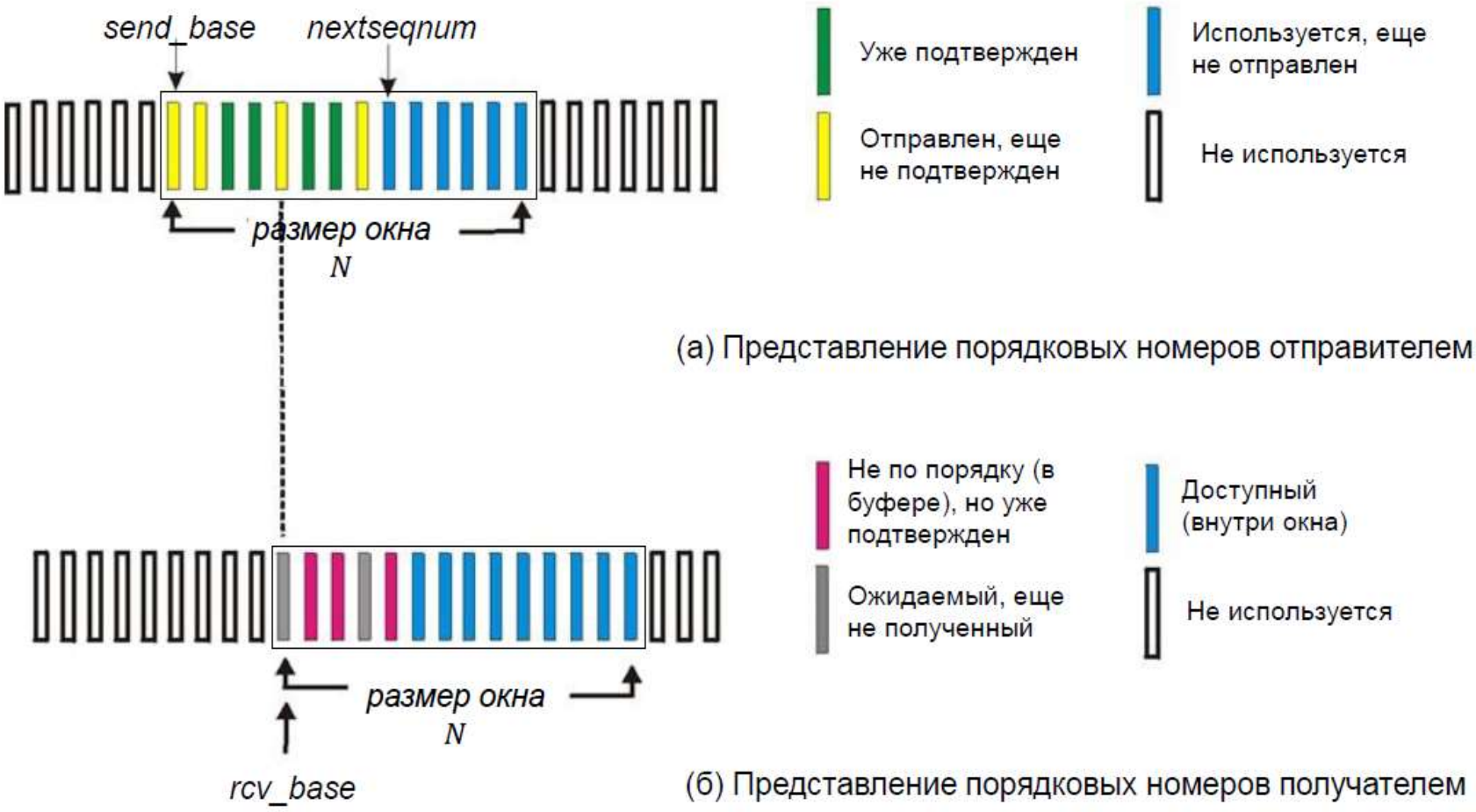
★ Выборочное повторение (SR – Selective Repeat)

Протокол устраняет ненужные повторные передачи данных

- Получатель отдельно подтверждает все корректно полученные пакеты
  - Буферы пакетов необходимы для упорядоченной доставки данных на верхний уровень
  - Отправитель повторно отправляет только пакеты, на которые не были получены ACK-подтверждения
  - Отправитель использует таймер для каждого неподтвержденного пакета

- Окно получателя:
  - **N** последовательных порядковых номеров
  - Ограничение количества порядковых номеров отправленных, но не подтвержденных пакетов

Выборочное повторение: окна отправителя и получателя



Выборочное повторение

Отправитель

Данные сверху:

- Если следующий доступный порядковый номер попадает в окно, отправить пакет

Таймаут(ы):

- Переслать пакет n, перезапустить таймер

ACK-пакет в диапазоне $[sendbase, sendbase+N]$:

- Отметить пакет n как полученный
- Если n – наименьший номер подтвержденного пакета, сдвинуть начало окна до следующего неподтвержденного пакета

Получатель

Пакет n в диапазоне $[rcvbase, rcvbase+N-1]$

- Отправить ACK(n)
- Если не по порядку, то добавить в буфер
- По порядку: доставить (также доставить все идущие по порядку пакеты из буфера), сдвинуть окно до следующего еще не подтвержденного пакета

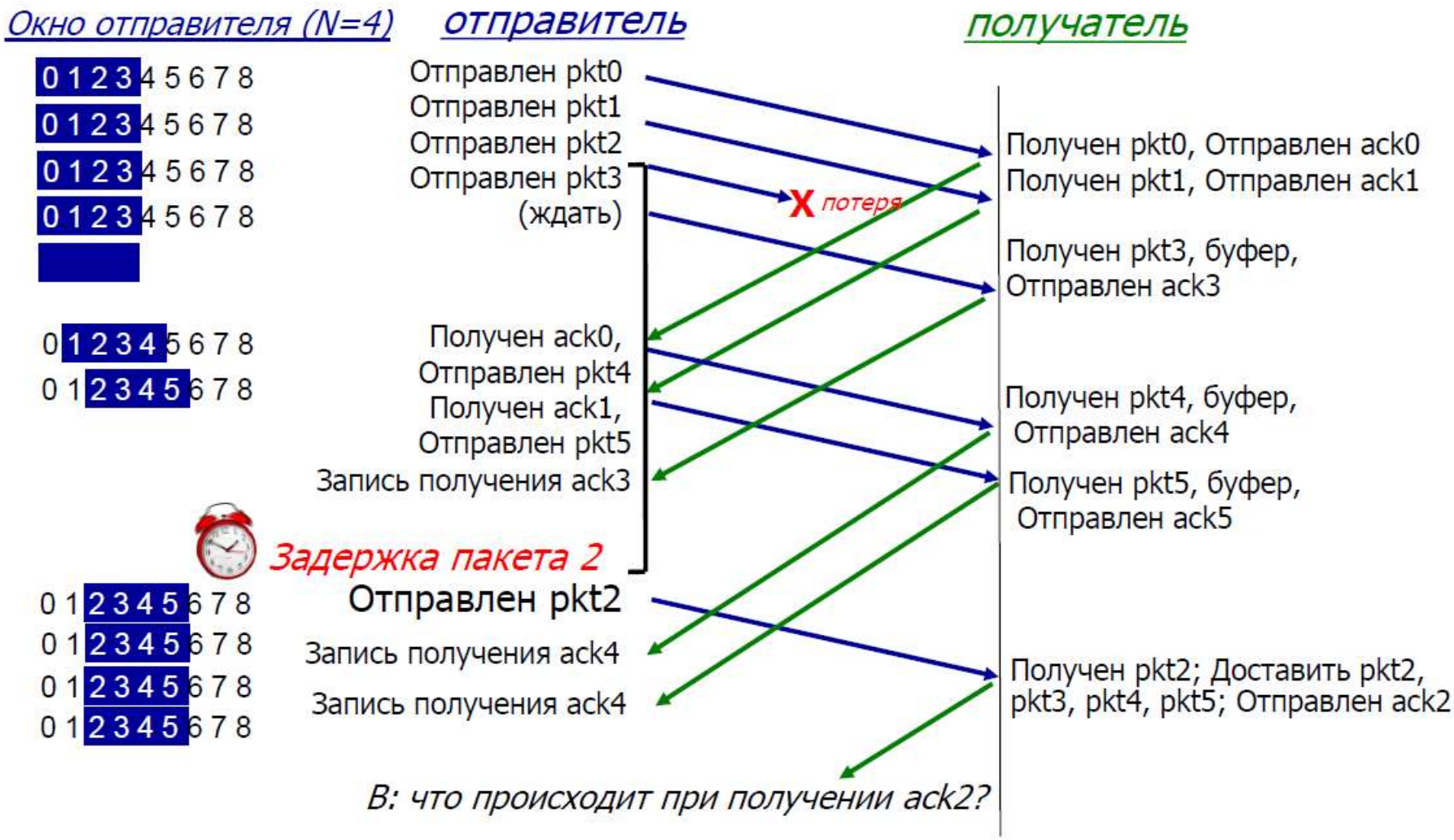
Пакет n в диапазоне $[rcvbase-N, rcvbase-1]$

- ACK(n)

Иначе:

- игнорировать

Выборочное повторение в действии



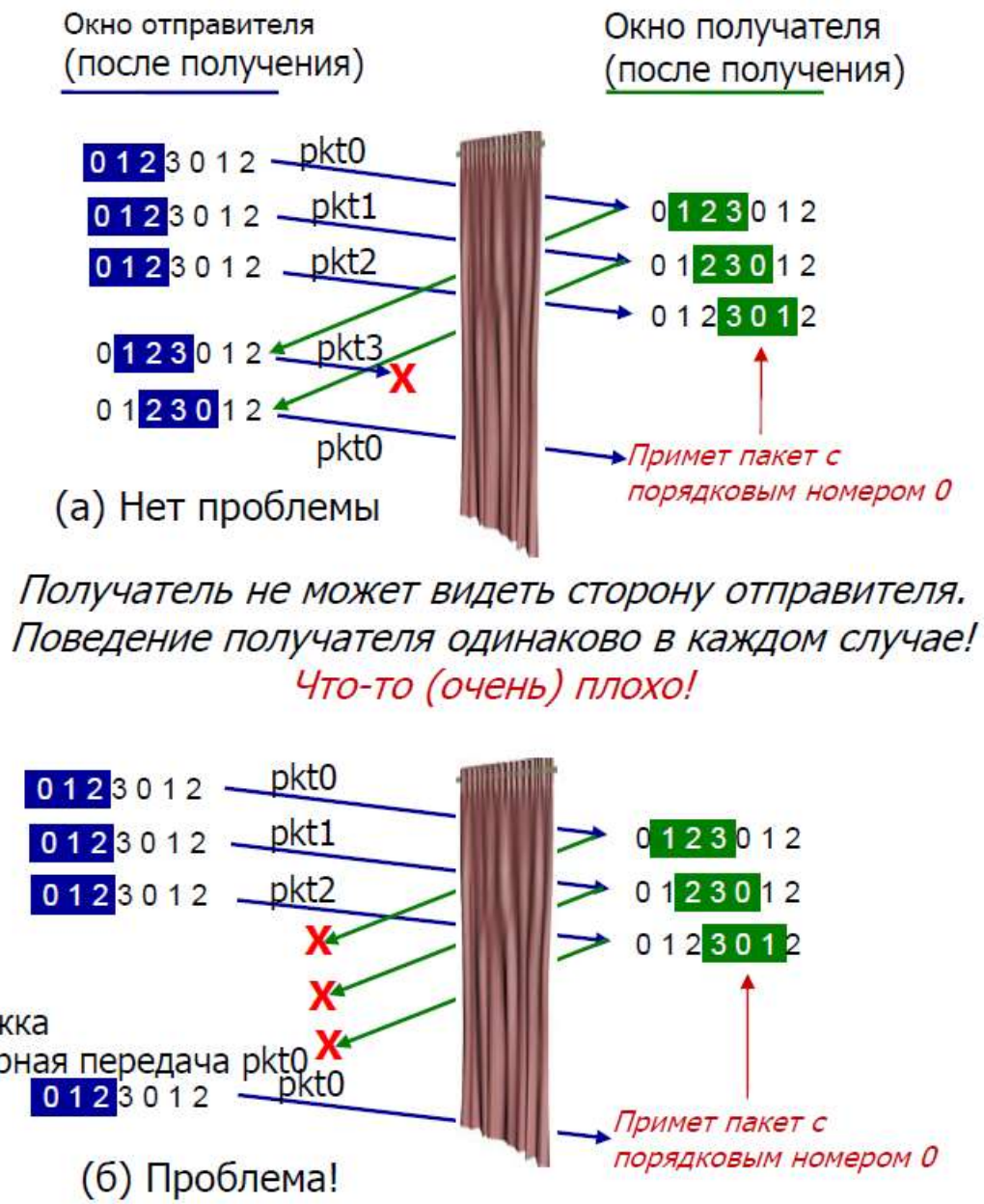
Факультет математики и компьютерных наук

Выборочное повторение:
проблема несоответствия окон

Пример:

- Порядковые номера: 0, 1, 2, 3
- Размер окна=3
- Получатель не видит разницы двух сценариев!
- Дублированные данные воспринимаются как новые

Вопрос: как отношение между величиной порядкового номера и размером окна приводит к проблеме, показанной на рисунке?



Передача данных с установлением логического соединения: TCP

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

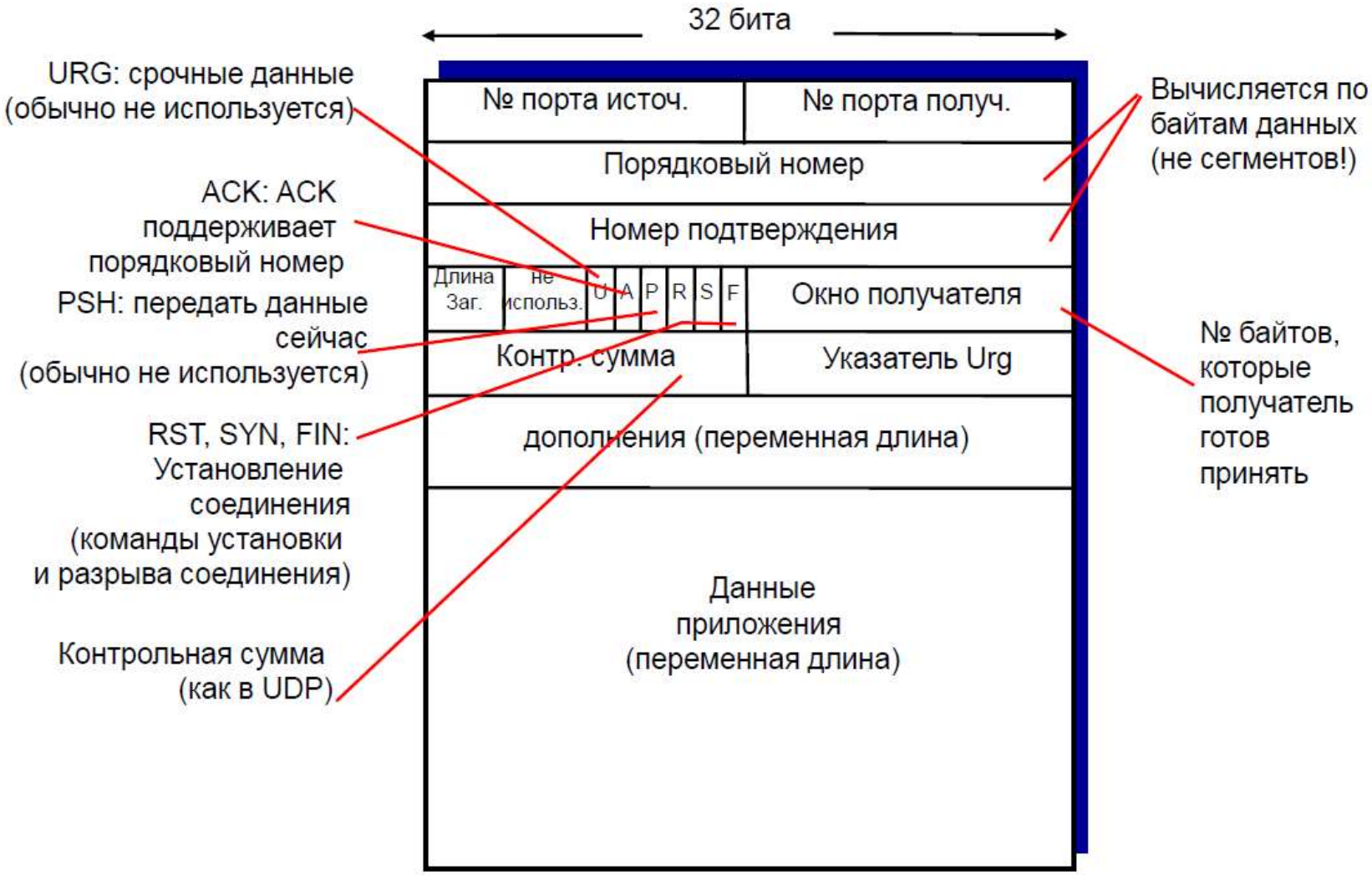
Протокол TCP, обзор RFC: 793,1122,1323, 2018, 2581

- Соединение точка-точка:
  - Один отправитель, один получатель
- Надежный упорядоченный поток байт:
  - нет «границ сообщений»
- Конвейеризация:
  - Управление перегрузкой и потоком TCP устанавливают размер окна

- TCP-соединение состоит из буферов, переменных и сокета соединения для обработки на одном хосте и другого набора буферов, переменных и сокета соединения для обработки на другом хосте

- Полнодуплексная передача данных:
  - Двунаправленный поток данных в одном соединении
  - MSS: максимальный размер сегмента
- С установлением логического соединения:
  - Установление соединения (обмен управляющими сообщениями) инициируется отправителем, состояние получателя до обмена данными
- Управляемый поток:
  - Отправитель не вызывает переполнение у получателя

Структура TCP-сегмента



Протокол TCP: порядковые номера, ACK-пакеты

Порядковые номера:

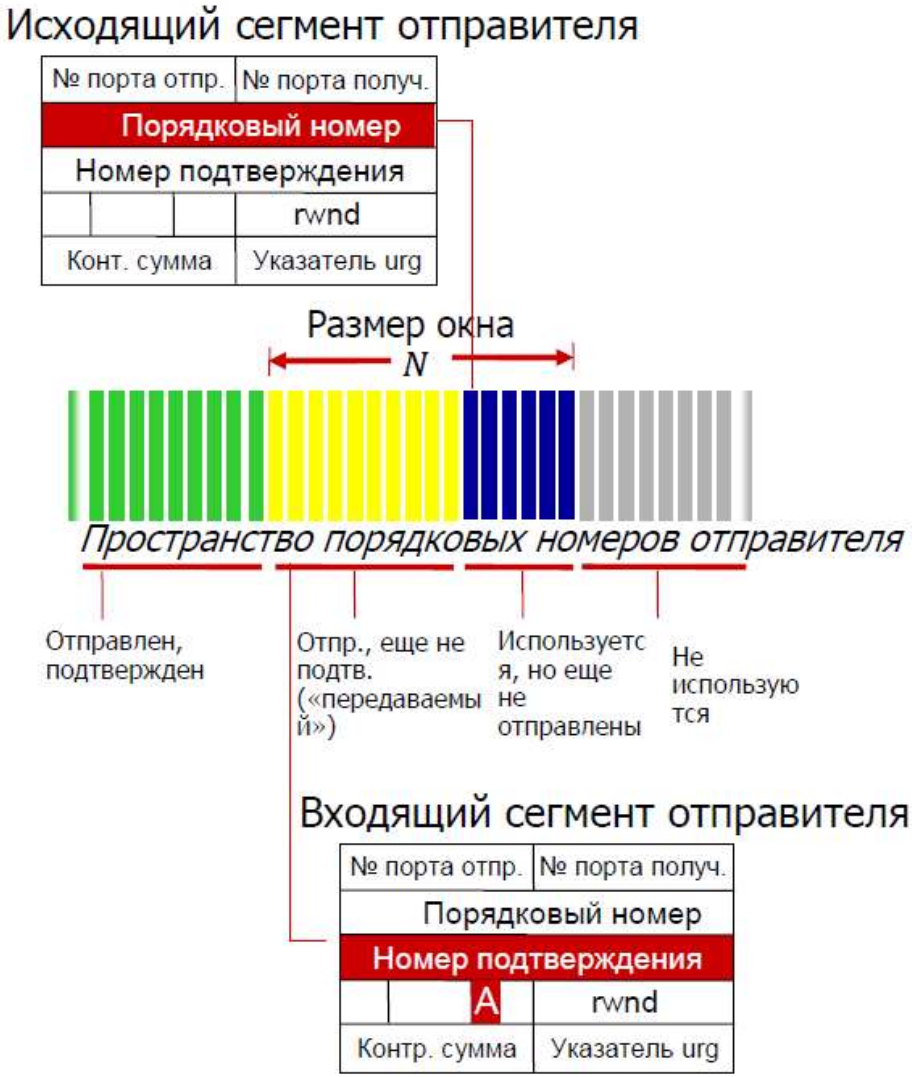
- «номер» первого байта данных сегмента в потоке байт

Подтверждения:

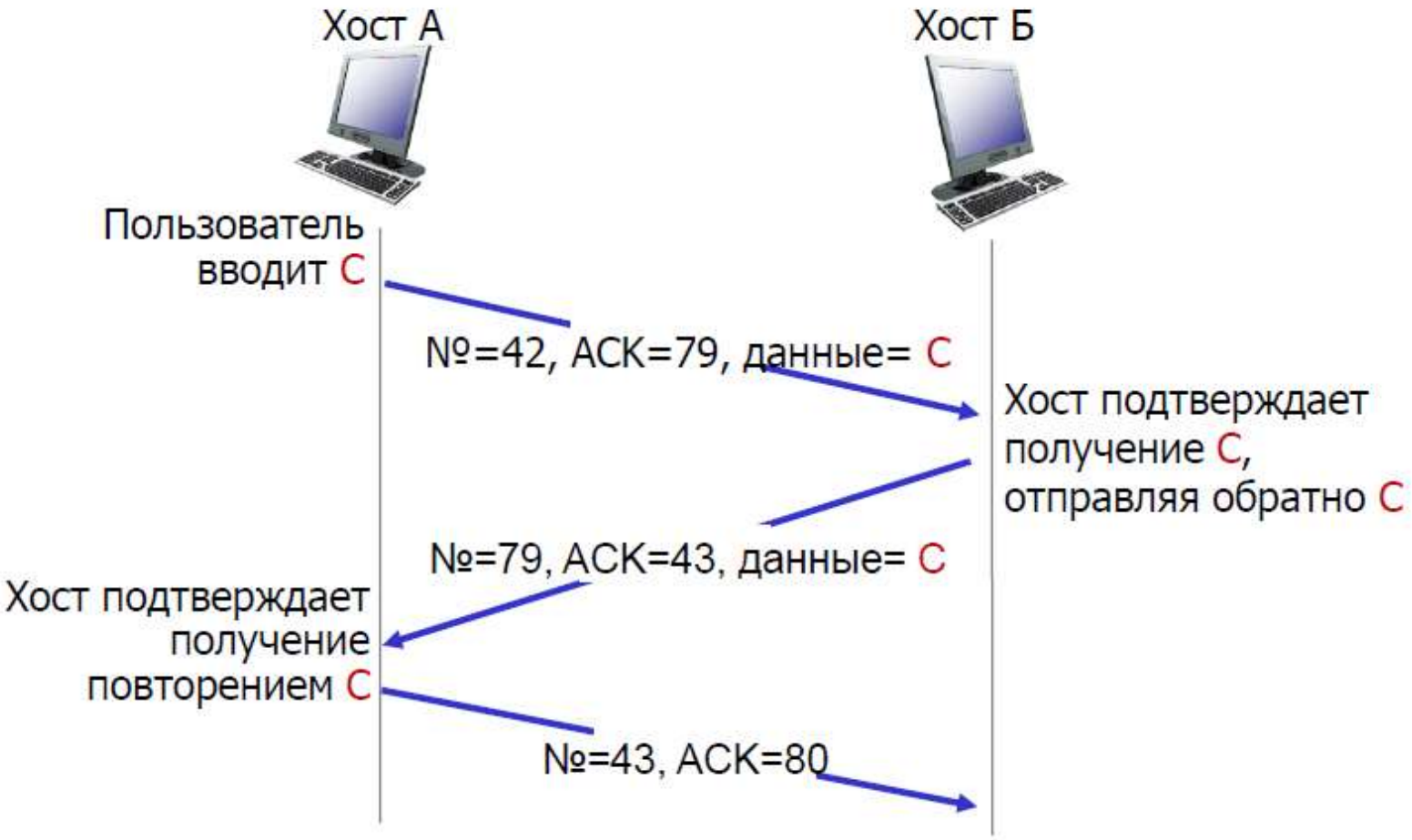
- порядковый номер следующего байта, ожидаемого от второй стороны

Вопрос: как получатель обрабатывает полученные не по порядку сегменты?

- Ответ: спецификация протокола TCP не регламентирует, ответ зависит от конкретной реализации



Протокол TCP: порядковые номера, ACK-пакеты



Простой пример telnet

Протокол TCP: время оборота, таймаут

Вопрос: как устанавливается значение таймаута в TCP?

- длиннее, чем RTT
  - но RTT изменяется
- Слишком короткий: преждевременный таймаут, ненужные повторные передачи
- Слишком долгий: медленная реакция на потерю сегментов

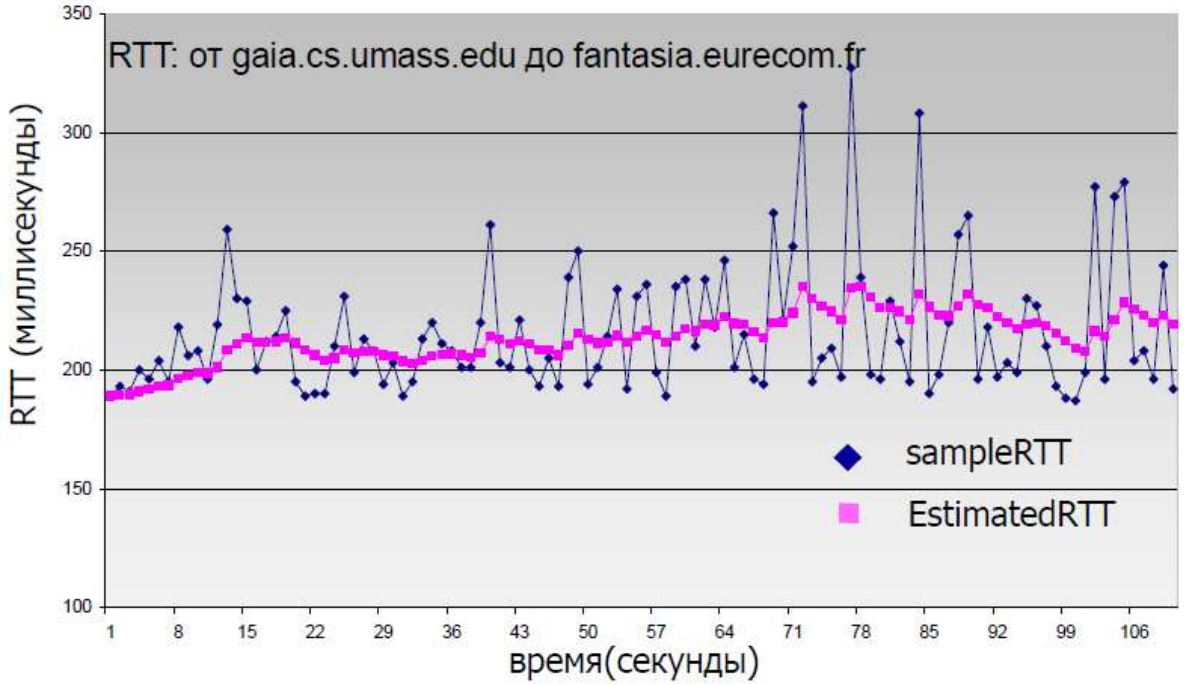
Вопрос: как вычисляется RTT?

- SampleRTT: измеряет время с момента передачи сегмента до получения ACK-пакета
  - Игнорирует повторные передачи
- SampleRTT будет меняться, желательно «сглаженное» измерение RTT
  - *Среднее нескольких* недавних измерений, а не только текущего SampleRTT

Протокол TCP: время оборота, таймаут

$$\boldsymbol{EstimatedRTT} = (1 - \alpha) \times \boldsymbol{EstimatedRTT} + \alpha \times \boldsymbol{SampleRTT}$$

- Экспоненциальное взвешенное скользящее среднее
- Влияние предыдущего значения уменьшается экспоненциально быстро
- Стандартное значение $\alpha = 0{,}125$



Факультет математики и компьютерных наук

Протокол TCP: время оборота, таймаут

- Интервал таймаута: EstimatedRTT + резерв
  - БОльшие изменения EstimatedRTT => бОльший резерв

- Оценка отклонения SampleRTT от EstimatedRTT:

$$\boldsymbol{DevRTT} = (1-\beta)\times \boldsymbol{DevRTT} + \beta \times |\boldsymbol{SampleRTT} - \boldsymbol{EstimatedRTT}|$$

(как правило, β=0,25)

$$\boldsymbol{TimeoutInterval} = \boldsymbol{EstimatedRTT} + 4 \times \boldsymbol{DevRTT}$$

Измеренное *RTT*

«резерв»

Протокол TCP: надежная передача данных

- Протокол TCP создает надежную службу поверх ненадежной службы протокола IP
  - Конвейеризация отправки сегментов
  - Общие подтверждения
  - Единственный таймер повторной передачи

- Повторная передача может быть вызвана:
  - Событиями таймаута
  - Дублирующими подтверждениями

- Для начала рассмотрим упрощенную версию TCP-отправителя:
  - Игнорирует дублирующие подтверждения
  - Игнорирует управление потоком и перегрузкой

Протокол TCP: события на стороне отправителя

Данные, полученные от приложения:

- создать сегмент с порядковым номером
- порядковый номер – номер первого байта данных в сегменте
- запустить таймер, если еще не был запущен
  - таймер для самого раннего (старого) неподтвержденного сегмента
  - интервал таймаута: TimeOutInterval

Таймаут:

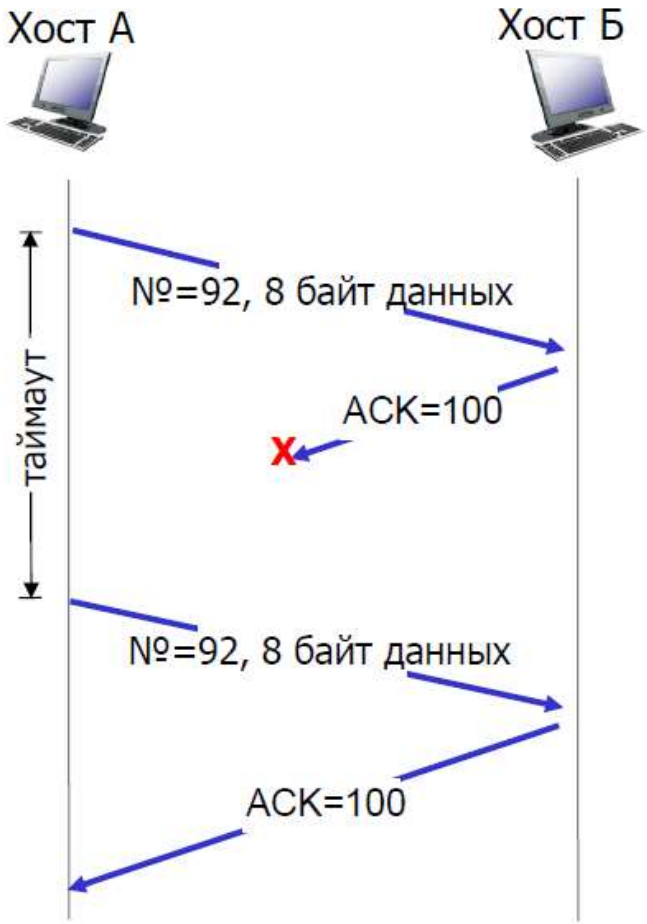
- повторно передать сегмент, который вызвал таймаут
- перезапустить таймер

Получено подтверждение:

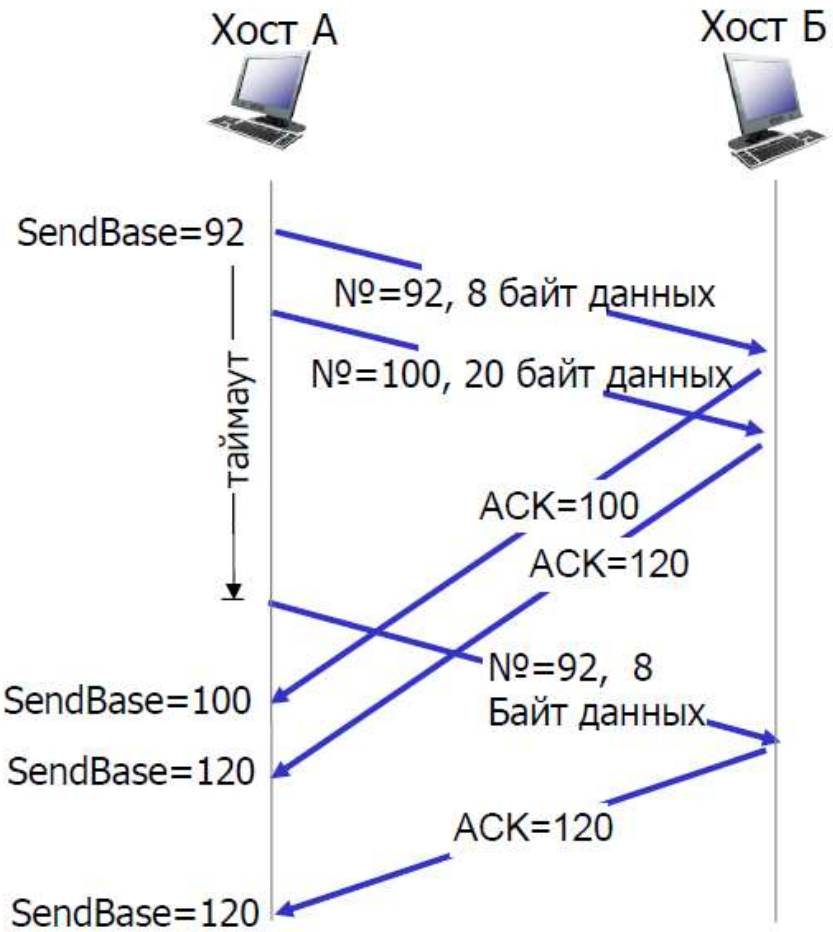
- если ACK-пакет подтверждает ранее неподтвержденные сегменты
  - обновить то, что уже подтверждено
  - перезапустить таймер, если еще есть неподтвержденные сегменты

Упрощенная FSM-схема TCP-отправителя

Протокол TCP: сценарии повторной передачи



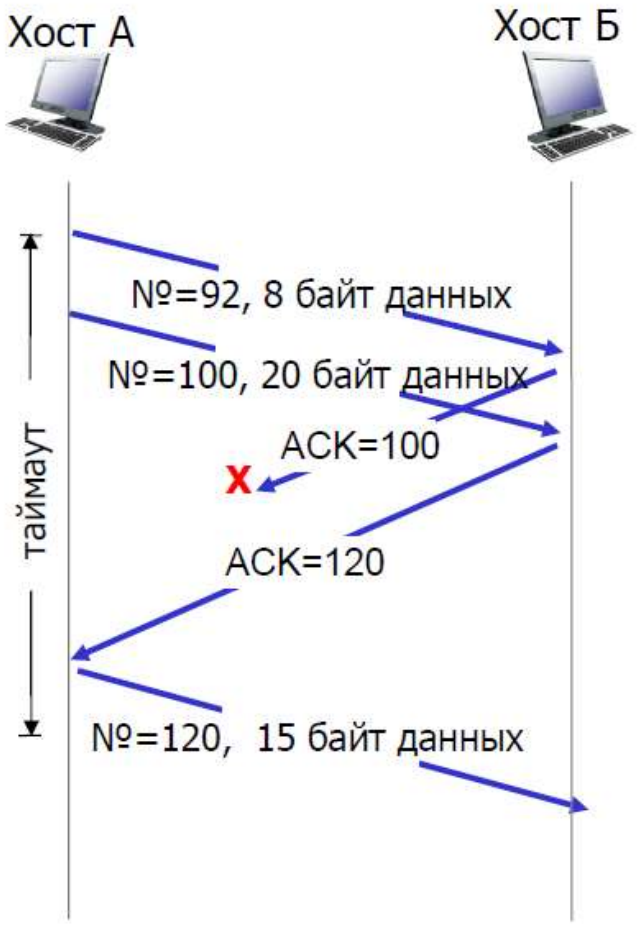
Сценарий потери ACK-пакета



Преждевременный таймаут

Факультет математики и компьютерных наук

Протокол TCP: сценарии повторной передачи



Общий ACK-пакет

Подтверждение на первый сегмент потеряно в сети, но непосредственно перед наступлением таймаута хост А получает пакет ACK с номером подтверждения 120

Хост А, таким образом, «понимает», что хост Б получил все данные до 119 байта, поэтому ни один из пакетов не будет отправлен повторно хостом А

Удвоение интервала

Каждый раз при наступлении таймаута протокол TCP повторно передает еще не подтвержденный сегмент с наименьшим порядковым номером

При каждой повторной передаче протокол TCP **удваивает значение интервала** времени ожидания по сравнению с предыдущим

Увеличение интервалов времени ожидания происходит экспоненциально при каждой повторной передаче

Но если таймер запускается по причине одного из двух других событий:

- получены данные от приложения верхнего уровня
- получен пакет ACK,

то значение TimeoutInterval определяется на основе последних значений EstimatedRTT и DevRTT

Ускоренная повторная передача

Используется механизм дублирования подтверждений, позволяющий передающей стороне обнаруживать потери пакетов до истечения интервала ожидания

Дублирующее подтверждение — это копия положительной квитанции **предыдущего** сегмента, отправляемая в ответ на получение следующего сегмента, если порядковый номер последнего превышает ожидаемый

Чтобы понять реакцию передающей стороны на появление дублирующих подтверждений, сначала необходимо выяснить причину, побуждающую к использованию такого механизма

Протокол TCP формирование ACK-пакета [RFC 1122, RFC 2581]

Событие получателя	*Действие TCP-получателя*
поступил сегмент в верном порядке с ожидаемым порядковым номером. Все данные до этого номера подтверждены	Задерживает ACK-пакет. Ожидает 500мс до следующего сегмента. Если следующего сегмента нет, отправляет ACK-пакет.
поступил сегмент в верном порядке с ожидаемым порядковым номером. Еще один сегмент ожидает отправку ACK-пакета	Незамедлительно отправляет общий ACK-пакет, подтверждая оба сегмента
Поступил сегмент в неверном порядке с порядковым номером превышающим Ожидаемый. Обнаружен разрыв	незамедлительно отправляет *дублирующий ACK*, указывает порядковый номер следующего ожидаемого сегмента
поступил сегмент, частично или полностью покрывающий разрыв	незамедлительно отправляет ACK-пакет, при условии, что сегмент соответствует нижнему краю разрыва

Протокол TCP: ускоренная повторная передача

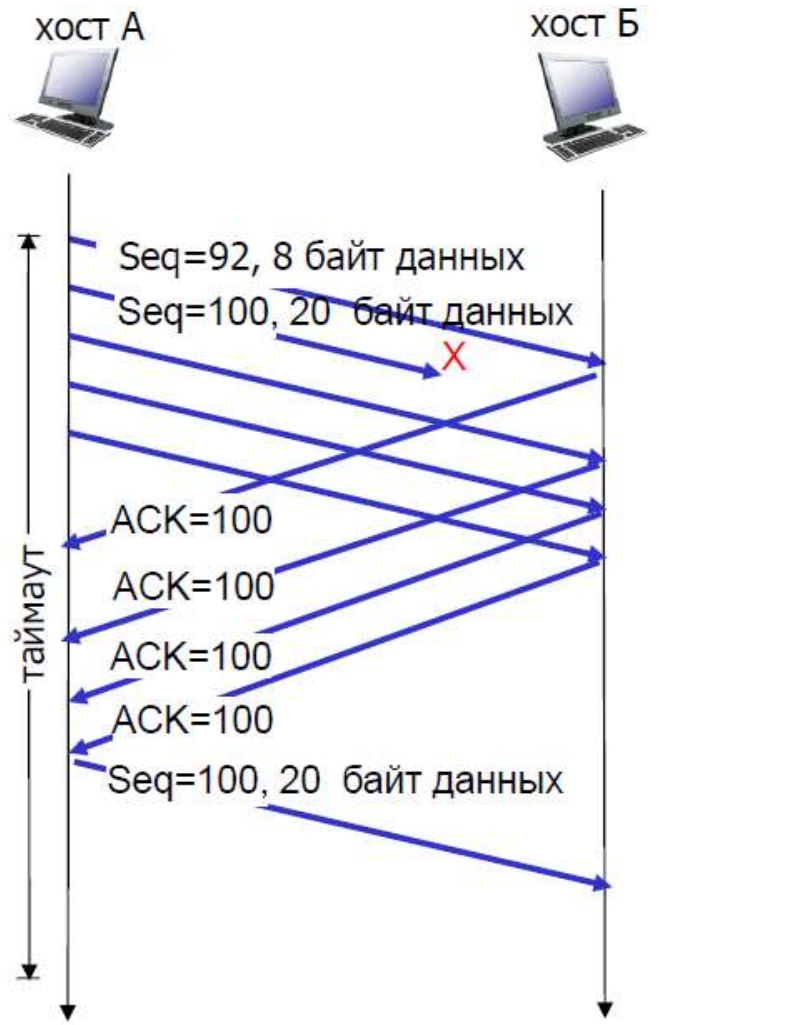
- Интервал ожидания (time out) часто относительно длинный:
  - Длительная задержка до повторной передачи потерянного пакета
- Обнаружение потери сегмента по дублирующим ACK-пакетам.
  - Отправитель часто последовательно передает несколько сегментов
  - Если сегмент потерян, это вызовет получение множества дублирующих ACK-пакетов

Ускоренная повторная передача TCP

Если отправитель получил 3 ACK-пакета на одни данные ("утроенный ACK-пакет"), повторно отправить неподтвержденный сегмент с меньшим порядковым номером

- То есть не дожидаться таймаута до повторной отправки потерянного сегмента

Протокол TCP: ускоренная повторная передача



Ускоренная повторная передача после получения отправителем трех дублирующих ACK-пакетов